

Universidad
Manuela Beltrán
Acreditada en Alta Calidad Multicampus

Service Worker: caché, modo sin conexión y estrategias de red

Informe de laboratorio. Guía n.º 5

Rodríguez Ángel, Santiago

Olaya Rojas, Santiago

Universidad Manuela Beltrán

Ingeniería de Software

Facultad de Ingeniería

Nuevas Tecnologías del Desarrollo

Grupo 6

Bogotá, Colombia

2026

Tabla de contenido

Introducción 5
1. Marco conceptual 6
1.1 Qué es un Service Worker 6
1.2 Ciclo de vida 6
1.3 Caché y estrategias de respuesta 7
1.4 Herramientas de Chrome DevTools 8
2. Desarrollo del laboratorio 9
2.1 Entorno y método de ejecución 9
2.2 Paso 1. Ciclo de vida y proyecto de prueba 10
2.3 Paso 2. Crear service-worker.js con Cache-First 11
2.4 Paso 3. Registrar el Service Worker y verificarlo en DevTools 14
2.5 Paso 4. Probar el modo sin conexión 17
2.6 Actualizar la versión del Service Worker 20
2.7 Desafío: Network-First 22
2.8 Reflexión: qué estrategia usar y por qué 29
Conclusiones 31
Referencias 32

Lista de tablas

Tabla 1 Datos de la práctica 5
Tabla 2 Dónde está cada elemento del entregable 6
Tabla 3 Eventos del ciclo de vida del Service Worker 7
Tabla 4 Estrategias de respuesta de un Service Worker 8
Tabla 5 Entorno de ejecución del laboratorio 9
Tabla 6 Archivos del proyecto 10
Tabla 7 Verificación del resultado esperado del paso 3 17
Tabla 8 Mediciones de la prueba sin conexión 20
Tabla 9 Diferencias entre las dos versiones del evento fetch 26
Tabla 10 Comparación de Cache-First y Network-First con el contenido cambiado en el servidor 26
Tabla 11 Estrategia elegida para cada caso 29

Lista de figuras

Figura 1 Estructura del proyecto en Visual Studio Code..... 11
Figura 2 Código de service-worker.js con Cache-First (parte 1 de 3)..... 12
Figura 3 Código de service-worker.js con Cache-First (parte 2 de 3)..... 13
Figura 4 Código de service-worker.js con Cache-First (parte 3 de 3)..... 14
Figura 5 Registro del Service Worker en app.js..... 15
Figura 6 Service Worker activo en Chrome DevTools..... 16
Figura 7 Mensajes del Service Worker y de la app en la consola..... 16
Figura 8 Cache Storage con el caché mi-pwa-v1..... 17
Figura 9 Casilla Offline marcada en el panel Service workers..... 18
Figura 10 Panel Network con Offline marcado: solicitudes atendidas por el Service Worker ... 18
Figura 11 La app sin conexión: recarga y nueva entrega..... 19
Figura 12 Versión nueva del Service Worker en espera..... 21
Figura 13 Cache Storage después de actualizar: solo queda mi-pwa-v2..... 22
Figura 14 Código de service-worker-network-first.js con Network-First (parte 1 de 3)..... 23
Figura 15 Código de service-worker-network-first.js con Network-First (parte 2 de 3)..... 24
Figura 16 Código de service-worker-network-first.js con Network-First (parte 3 de 3)..... 25
Figura 17 Página tras cambiar el contenido en el servidor: Cache-First y Network-First..... 27
Figura 18 Página con el servidor detenido: Cache-First y Network-First..... 28

Introducción

Este informe documenta la guía de laboratorio n.º 5, «Service Worker», del módulo 2 (PWA) de Nuevas Tecnologías del Desarrollo. El objetivo de la guía es implementar un Service Worker funcional que permita que una aplicación web siga funcionando sin conexión, gestione el caché de sus recursos e intercepte las peticiones de red (Universidad Manuela Beltrán [UMB], 2026).

Se construyó una aplicación pequeña, «Agenda de entregas UMB», y se le agregó el Service Worker de la guía con estrategia Cache-First. Después se registró en la aplicación, se verificó en Chrome DevTools, se probó sin conexión y se resolvió el desafío: reescribir el evento *fetch* con Network-First y comparar las dos estrategias con una medición. El informe cierra con la reflexión sobre qué estrategia conviene para un blog de noticias, una calculadora y un chat en tiempo real.

El título y el objetivo de la guía mencionan las notificaciones push, pero ninguno de sus pasos ni de sus criterios de evaluación las pide; por eso no se implementaron. Todas las capturas son de ejecuciones reales; la sección 2.1 explica el entorno y cómo se operó. La Tabla 1 reúne los datos de la práctica y la Tabla 2 indica dónde está cada elemento del entregable.

Tabla 1
Datos de la práctica

Campo	Valor
Grupo	Grupo 6
Integrante 1	Rodríguez Ángel, Santiago
Integrante 2	Olaya Rojas, Santiago
Fecha de ejecución	19 de octubre de 2026
Asignatura	Nuevas Tecnologías del Desarrollo, módulo 2: PWA
Guía	Guía de laboratorio n.º 5: Service Worker (offline, caché, fetch)

Nota. Elaboración propia con base en la guía (UMB, 2026).

Tabla 2
Dónde está cada elemento del entregable

Elemento del entregable	Dónde está
1. Carpeta .zip con todo el proyecto, con service-worker.js funcional	Archivo Lab5_ServiceWorker_Grupo6.zip, carpeta proyecto; estructura en la Figura 1
2. Captura del Service Worker activo en DevTools	Figura 6 y Figura 7, sección 2.4
3. Captura de Cache Storage con los archivos	Figura 8, sección 2.4
4. Captura de la app funcionando sin conexión	Figura 9, Figura 10 y Figura 11, sección 2.5
5. Ambas versiones del Service Worker, comentadas	Secciones 2.3 y 2.7 (Figura 2 a Figura 4 y Figura 14 a Figura 16); archivos service-worker.js y service-worker-network-first.js
6. Respuesta escrita a la reflexión sobre estrategias de caché	Sección 2.8 y Tabla 11

Nota. Elaboración propia con base en el entregable de la guía (UMB, 2026).

1. Marco conceptual

1.1 Qué es un Service Worker

Un Service Worker es un intermediario dirigido por eventos que se sitúa entre la aplicación web, el navegador y la red cuando esta está disponible. Permite interceptar y modificar las peticiones, guardar recursos en caché y ofrecer una experiencia sin conexión (MDN Web Docs, s. f.-a). Corre en un contexto de trabajador, en un hilo distinto al de la página, por lo que no accede al DOM, y solo está disponible en contextos seguros: se sirve por HTTPS, y *http://localhost* también está permitido para desarrollar (MDN Web Docs, s. f.-a, s. f.-b).

1.2 Ciclo de vida

El Service Worker de la guía usa tres eventos, que resume la Tabla 3. A ellos se suma una fase intermedia que explica la sección 2.6: cuando se instala una versión nueva, esta espera hasta que ninguna página use la anterior, de modo que solo una versión controle las páginas a la vez (Archibald, 2016).

Tabla 3

Eventos del ciclo de vida del Service Worker

Evento	Cuándo ocurre	Qué se hace aquí
install	La primera vez que se registra el Service Worker, y con cada versión nueva.	Guardar los archivos en caché (pre-caching). Si la promesa de *waitUntil* falla, la instalación falla (MDN Web Docs, s. f.-b).
waiting	Después de instalar una versión nueva, mientras la anterior controle páginas.	Nada: la versión nueva espera; *skipWaiting()* permite saltar la espera (Archibald, 2016).
activate	Cuando la versión anterior ya no controla páginas.	Eliminar los cachés viejos y migrar datos (Archibald, 2016).
fetch	Cada vez que la app hace una petición HTTP.	Interceptar la petición y responder desde el caché o desde la red, con *event.respondWith()*.

Nota. Elaboración propia con base en la guía (UMB, 2026), MDN Web Docs (s. f.-b) y Archibald (2016).

1.3 Caché y estrategias de respuesta

La API Cache guarda pares petición-respuesta en cachés con nombre. *caches.open()* crea o abre uno, *cache.addAll()* guarda varios recursos a la vez, *cache.put()* guarda uno, y *caches.match()* devuelve la respuesta guardada o *undefined* si no hay copia (MDN Web Docs, s. f.-b). Una respuesta es un flujo que se lee una sola vez, por eso, para guardarla y a la vez entregarla a la página, se usa *clone()*: una copia va al caché y la original va al navegador (MDN Web Docs, s. f.-b).

Sobre esa API se construyen las estrategias de la Tabla 4. Las dos que pide la guía son Cache-First, que consulta primero el caché, y Network-First, que consulta primero la red.

Tabla 4

Estrategias de respuesta de un Service Worker

Estrategia	Cómo responde	Para qué contenido conviene
Cache-First (caché y, si no hay, red)	Devuelve la copia del caché; solo si no existe va a la red. Los cambios del servidor no se ven mientras haya copia.	Aplicaciones que funcionan primero sin conexión y recursos estáticos, sobre todo los versionados con un identificador en el nombre (Archibald, 2014; Chrome for Developers, 2021).
Network-First (red y, si falla, caché)	Pide a la red; si responde, guarda una copia y la entrega; si falla, entrega la última copia.	Contenido que cambia a menudo, como artículos o respuestas de una API. Con una red lenta o intermitente, el usuario espera mucho antes de ver la copia (Archibald, 2014; Chrome for Developers, 2021).
Stale-while-revalidate	Entrega la copia del caché de inmediato y actualiza el caché en segundo plano para la próxima visita.	Contenido que debe mantenerse razonablemente al día pero no es crítico, como los avatares (Archibald, 2014; Chrome for Developers, 2021).
Solo red (*network only*)	Siempre va a la red, sin alternativa sin conexión.	Lo que no tiene equivalente sin conexión, como las peticiones que no son GET (Archibald, 2014).

Nota. Elaboración propia con base en Archibald (2014) y Chrome for Developers (2021).

1.4 Herramientas de Chrome DevTools

El panel Application de Chrome DevTools tiene un panel «Service workers» con el estado de cada Service Worker y tres casillas: *Offline* pone a DevTools en modo sin conexión, *Update on reload* actualiza el Service Worker en cada carga y *Bypass for network* hace que las peticiones salten el Service Worker y vayan directo a la red. Más abajo, «Cache storage» muestra de solo lectura los recursos que el Service Worker guardó con la API Cache (Basques y Emelianova, 2016). Esas son las herramientas con las que se verifica el laboratorio.

2. Desarrollo del laboratorio

2.1 Entorno y método de ejecución

La guía propone abrir la app con Live Server de Visual Studio Code y usar Chrome. Como equivalente se usó un servidor web local mínimo, escrito en Python, que sirve la carpeta del proyecto en *http://localhost:8095*; *localhost* es un origen seguro, así que el Service Worker funciona sin HTTPS (MDN Web Docs, s. f.-b). El servidor envía el encabezado *Cache-Control: no-store* para que la caché HTTP del navegador nunca oculte lo que hace, o deja de hacer, el Service Worker.

Las pruebas las ejecutó un script que controla Google Chrome mediante el protocolo Chrome DevTools. Chrome sirve su propia interfaz de DevTools; el script la abrió conectada a la pestaña de la app, de modo que los paneles de las capturas son los reales de Chrome, y los clics sobre ellos, como marcar «Offline», son eventos de entrada reales. La excepción es la actualización de versión de la sección 2.6, que se activó con el comando *ServiceWorker.skipWaiting* del mismo protocolo, equivalente al botón *skipWaiting* que muestra DevTools. La Tabla 5 resume el entorno.

Tabla 5

Entorno de ejecución del laboratorio

Parámetro	Valor
Fecha de ejecución	19 de octubre de 2026
Navegador	Google Chrome 155.0.8059.84, en modo sin ventana (*headless*), controlado por script
Servidor local	Python (*http.server*) en http://localhost:8095, con *Cache-Control: no-store*
Editor	Visual Studio Code 1.142.0, con un perfil aislado y sin extensiones
Script de pruebas	herramientas/ejecutar_laboratorio.mjs, con Node.js 24.14.0
Resultados medidos	evidencia/resultados.json, escrito por el script con cada medición

Nota. Elaboración propia con datos de la ejecución en Chrome (Google, s. f.) y Visual Studio Code (Microsoft, s. f.).

2.2 Paso 1. Ciclo de vida y proyecto de prueba

El primer paso de la guía es conceptual: los tres eventos del ciclo de vida (sección 1.2). Para poder probarlos se creó una aplicación completa, «Agenda de entregas UMB», una lista de entregas pendientes que se guarda en el navegador (*localStorage*) y muestra si hay conexión. La página lleva además un renglón «Versión del contenido» que sirve de testigo en las pruebas de las secciones 2.6 y 2.7: cambiarlo en el servidor permite ver qué versión entrega cada estrategia. La Tabla 6 lista los archivos y la Figura 1 muestra el proyecto abierto en Visual Studio Code.

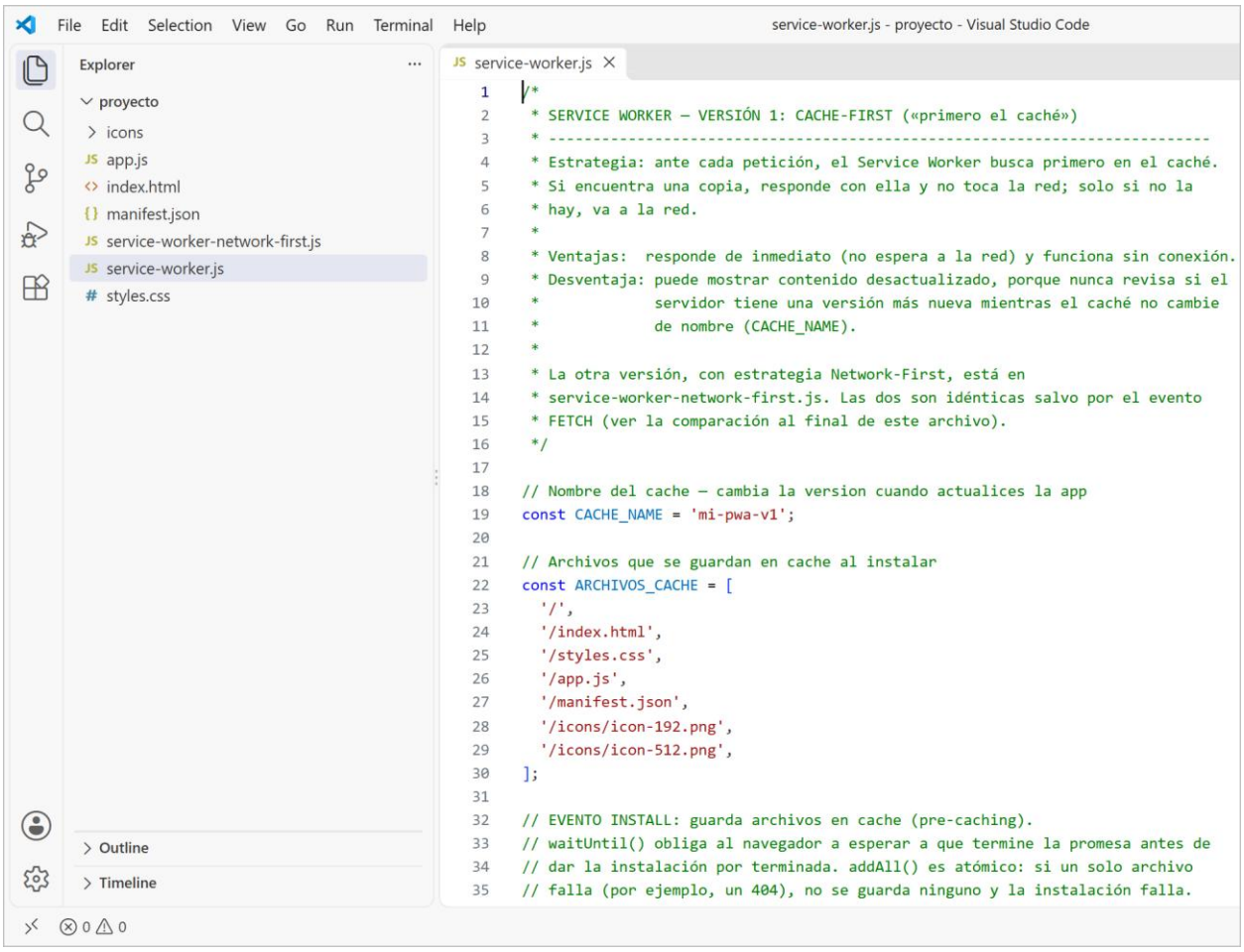
Tabla 6
Archivos del proyecto

Archivo	Función
index.html	Página única de la app: encabezado con la versión del contenido y el estado de la red, formulario y lista de entregas.
styles.css	Estilos de la app; los colores y las medidas están definidos como variables al inicio del archivo.
app.js	Lógica de la lista (agregar, marcar, quitar), indicador de conexión y, al final, el registro del Service Worker.
manifest.json	Manifiesto de la app: nombre, colores, modo de visualización e íconos de 192 y 512 píxeles.
icons/icon-192.png, icons/icon-512.png	Íconos de la app; también se guardan en el caché.
service-worker.js	Service Worker con estrategia Cache-First, tal como lo entrega la guía, con comentarios agregados.
service-worker-network-first.js	Versión del desafío, con estrategia Network-First; solo cambia el evento *fetch*.

Nota. Elaboración propia.

Figura 1

Estructura del proyecto en Visual Studio Code



Nota. Captura real del explorador de Visual Studio Code con la carpeta proyecto y service-worker.js abierto.

2.3 Paso 2. Crear service-worker.js con Cache-First

Se creó *service-worker.js* en la raíz del proyecto con el código de la guía, sin cambiar su lógica, y se le agregaron comentarios en español que explican cada parte. Figura 2 a Figura 4 muestran el archivo completo. El encabezado resume la estrategia, el nombre del caché (*mi-pwa-v1*) cambia cuando se actualiza la app, y la lista *ARCHIVOS_CACHE* reúne los siete recursos que se guardan al instalar.

Figura 2

Código de service-worker.js con Cache-First (parte 1 de 3)

service-worker.js

```
/*
 * SERVICE WORKER — VERSIÓN 1: CACHE-FIRST («primero el caché»)
 * ------------------------------------------------------------------------
 * Estrategia: ante cada petición, el Service Worker busca primero en el caché.
 * Si encuentra una copia, responde con ella y no toca la red; solo si no la
 * hay, va a la red.
 *
 * Ventajas:  responde de inmediato (no espera a la red) y funciona sin conexión.
 * Desventaja: puede mostrar contenido desactualizado, porque nunca revisa si el
 *             servidor tiene una versión más nueva mientras el caché no cambie
 *             de nombre (CACHE_NAME).
 *
 * La otra versión, con estrategia Network-First, está en
 * service-worker-network-first.js. Las dos son idénticas salvo por el evento
 * FETCH (ver la comparación al final de este archivo).
 */

// Nombre del cache — cambia la version cuando actualices la app
const CACHE_NAME = 'mi-pwa-v1';

// Archivos que se guardan en cache al instalar
const ARCHIVOS_CACHE = [
  '/',
  '/index.html',
  '/styles.css',
  '/app.js',
  '/manifest.json',
  '/icons/icon-192.png',
  '/icons/icon-512.png',
];

```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker.js, líneas 1 a 31.

En *install*, *waitUntil()* obliga al navegador a esperar a que termine la promesa antes de dar la instalación por terminada, y *addAll()* es atómico: si un solo archivo falla, por ejemplo con un error 404, no se guarda ninguno y la instalación falla (MDN Web Docs, s. f.-b).

Figura 3

Código de service-worker.js con Cache-First (parte 2 de 3)

```
32  // EVENTO INSTALL: guarda archivos en cache (pre-caching).
33  // waitUntil() obliga al navegador a esperar a que termine la promesa antes de
34  // dar la instalación por terminada. addAll() es atómico: si un solo archivo
35  // falla (por ejemplo, un 404), no se guarda ninguno y la instalación falla.
36  self.addEventListener('install', (event) => {
37    console.log('[SW] Instalando...');
38    event.waitUntil(
39      caches.open(CACHE_NAME).then((cache) => {
40        console.log('[SW] Guardando archivos en cache');
41        return cache.addAll(ARCHIVOS_CACHE);
42      })
43    );
44  });
45
46  // EVENTO ACTIVATE: limpia caches viejos.
47  // Borra todos los cachés cuyo nombre no sea el actual; así, al subir de
48  // 'mi-pwa-v1' a 'mi-pwa-v2', el caché anterior no queda ocupando espacio.
49  self.addEventListener('activate', (event) => {
50    console.log('[SW] Activando...');
51    event.waitUntil(
52      caches.keys().then((nombres) => {
53        return Promise.all(
54          nombres.filter(n => n !== CACHE_NAME).map(n => caches.delete(n))
55        );
56      })
57    );
58  });
59
```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker.js, líneas 32 a 59.

En *activate*, el Service Worker borra todos los cachés cuyo nombre no sea el actual; así, al subir la versión del nombre, el caché anterior desaparece. En *fetch*, *caches.match()* busca la petición: si hay copia la devuelve sin tocar la red, y si no, la pide con *fetch()*.

Figura 4

Código de service-worker.js con Cache-First (parte 3 de 3)

```
60  // EVENTO FETCH: Cache-First (primero cache, si no hay, va a la red).
61  // respondWith() decide qué respuesta recibe la página. caches.match() busca la
62  // petición en todos los cachés; devuelve undefined si no hay copia.
63  self.addEventListener('fetch', (event) => {
64    event.respondWith(
65      caches.match(event.request).then((respuestaCache) => {
66        if (respuestaCache) {
67          return respuestaCache; // responde desde cache
68        }
69        return fetch(event.request); // va a la red
70      })
71    );
72  });
73
74  /*
75   * COMPARACIÓN CON NETWORK-FIRST (service-worker-network-first.js)
76   * ------------------------------------------------------------------------
77   *                       Cache-First (este archivo)   Network-First
78   * Primer intento        caché                        red
79   * Si el primero falla   red                          caché
80   * Contenido             puede estar desactualizado   siempre el más reciente
81   * Velocidad             inmediata                    depende de la red
82   * Sin conexión          funciona                     funciona con lo ya guardado
83   * Actualiza el caché    solo al instalar             en cada respuesta de la red
84   * Conviene para         recursos que casi no cambian contenido que cambia a menudo
85   */
86
```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker.js, líneas 60 a 86. Al final del archivo, un cuadro compara las dos estrategias.

2.4 Paso 3. Registrar el Service Worker y verificarlo en DevTools

Al final de *app.js* se agregó el bloque de registro de la guía. Registra el Service Worker solo si el navegador lo admite y espera al evento *load* para no competir con la carga de la página (Figura 5).

Figura 5

Registro del Service Worker en app.js

```
// Registrar el Service Worker
if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
        navigator.serviceWorker.register('/service-worker.js')
            .then((registro) => {
                console.log('[App] Service Worker registrado:', registro.scope);
            })
            .catch((error) => {
                console.error('[App] Error al registrar SW:', error);
            });
    });
} else {
    console.log('[App] Service Workers no soportados en este navegador');
}

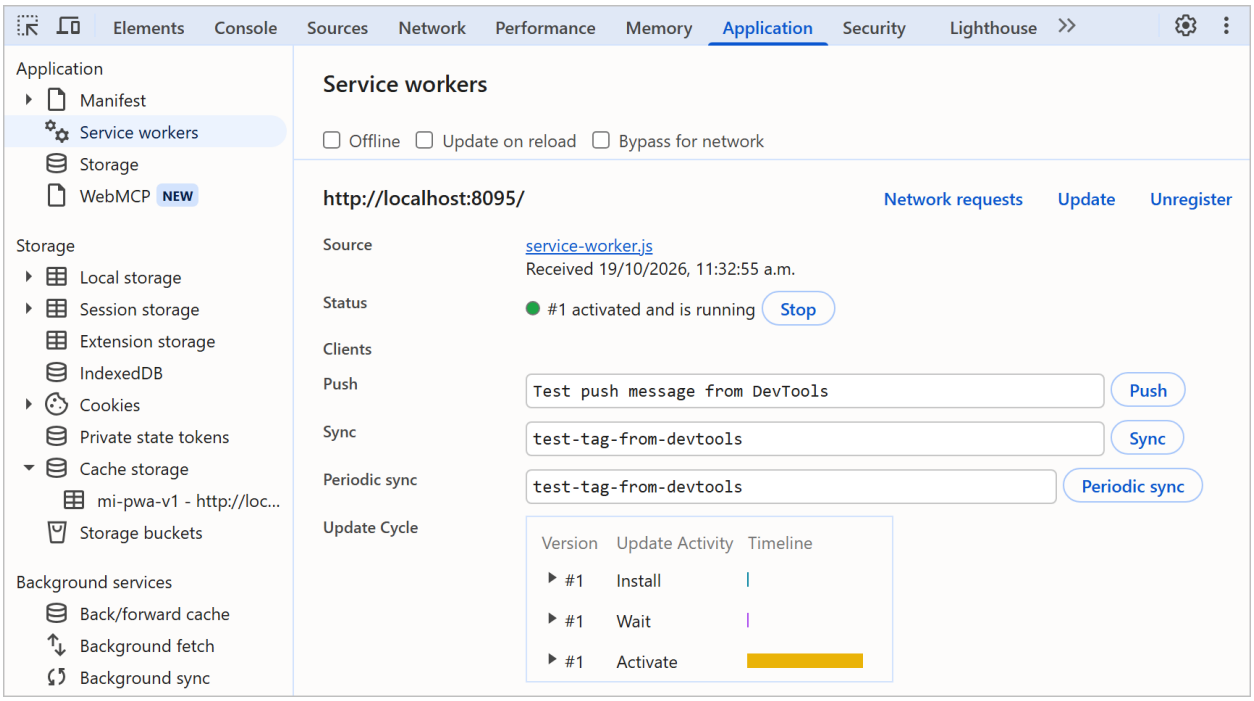
```

Nota. Captura real de Visual Studio Code 1.142.0: app.js, líneas 97 a 110.

Con la app cargada, el panel Application > Service workers muestra el Service Worker como «activated and is running», con el punto verde, para el origen http://localhost:8095/ (Figura 6). La consola registra los cuatro mensajes: el de la app y los tres del Service Worker (Figura 7), y Cache storage muestra el caché *mi-pwa-v1* con los siete archivos de *ARCHIVOS_CACHE* (Figura 8).

Figura 6

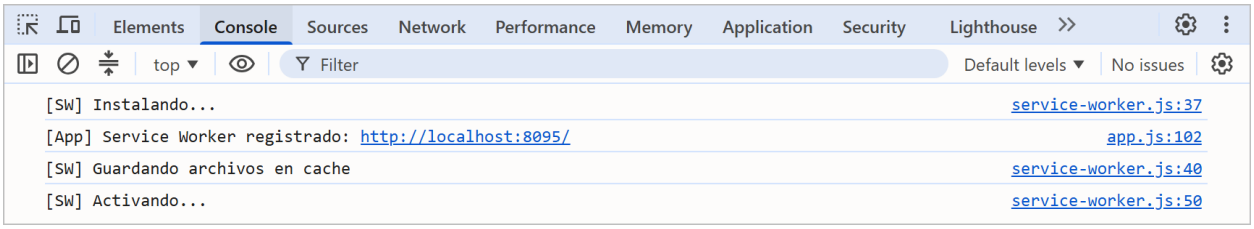
Service Worker activo en Chrome DevTools



Nota. Captura real de Chrome DevTools, Application > Service workers. Se recortó el espacio vacío bajo el cuadro «Update Cycle».

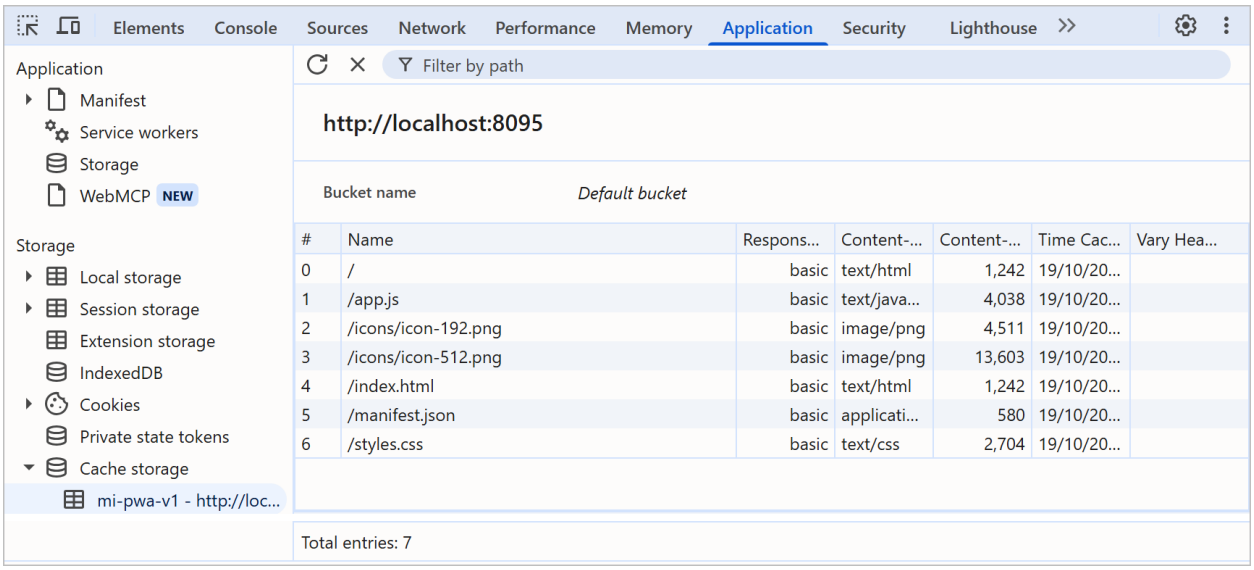
Figura 7

Mensajes del Service Worker y de la app en la consola



Nota. Captura real de la consola de Chrome DevTools, sin errores. A la derecha de cada mensaje aparece el archivo y la línea que lo emitió.

Figura 8
Cache Storage con el caché mi-pwa-v1



Nota. Captura real de Chrome DevTools, Application > Cache storage. Se unió la franja inferior con el total de entradas al recorte de la tabla, y se dejó entre ambas un pequeño espacio en blanco.

Tabla 7
Verificación del resultado esperado del paso 3

Resultado esperado por la guía	Resultado observado	Evidencia
Service Worker activo y corriendo (verde en DevTools)	«#1 activated and is running», punto verde.	Figura 6
Mensajes [SW] Instalando... y [SW] Activando... en la consola	Aparecen los cuatro mensajes, sin errores.	Figura 7
Cache Storage con todos los archivos de ARCHIVOS_CACHE	mi-pwa-v1 con 7 entradas: las 7 de la lista.	Figura 8

Nota. Elaboración propia con base en la guía (UMB, 2026) y en la ejecución.

Un detalle que la medición confirmó: la primera carga de la página no queda controlada por el Service Worker, solo las siguientes. En la primera carga, *navigator.serviceWorker.controller* fue nulo; tras una recarga, ya apuntaba al Service Worker (sección 2.5). Se debe a que el código de la guía no llama a *clients.claim()*, que es lo que permite a un Service Worker recién activado tomar el control de páginas cargadas sin él (Archibald, 2016).

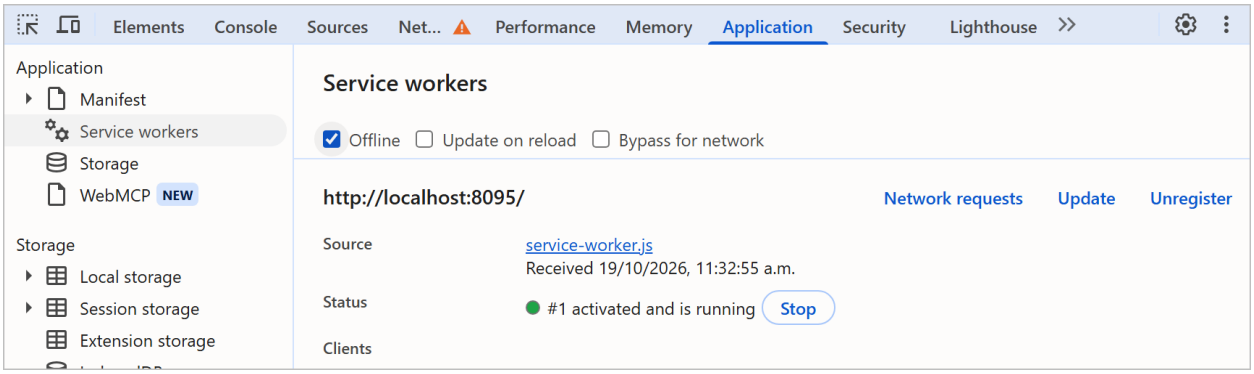
2.5 Paso 4. Probar el modo sin conexión

Siguiendo el procedimiento de la guía, en Application > Service workers se marcó la casilla «Offline» (Figura 9) y se recargó la página. En el panel Network, las seis solicitudes de la carga,

con el documento, la hoja de estilos, el script, el manifiesto y el ícono (dos veces), tuvieron estado 200 y tamaño «(ServiceWorker)», con 0 B transferidos (Figura 10). La app cargó completa (Figura 11a).

Figura 9

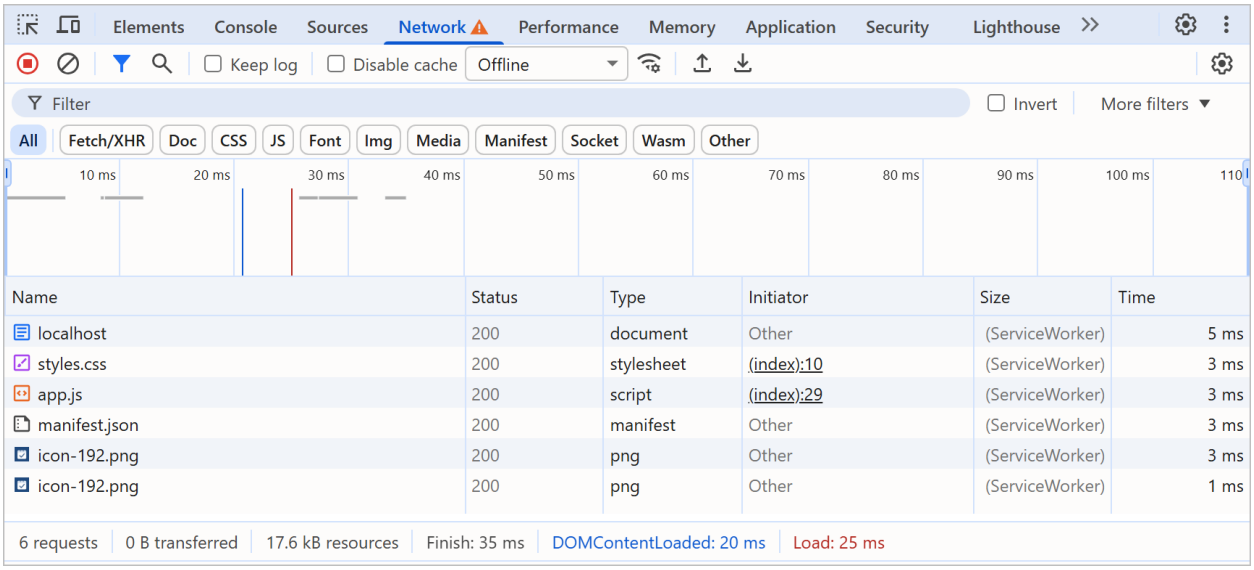
Casilla Offline marcada en el panel Service workers



Nota. Captura real de Chrome DevTools. La pestaña Network muestra una advertencia porque la limitación de red está activa.

Figura 10

Panel Network con Offline marcado: solicitudes atendidas por el Service Worker



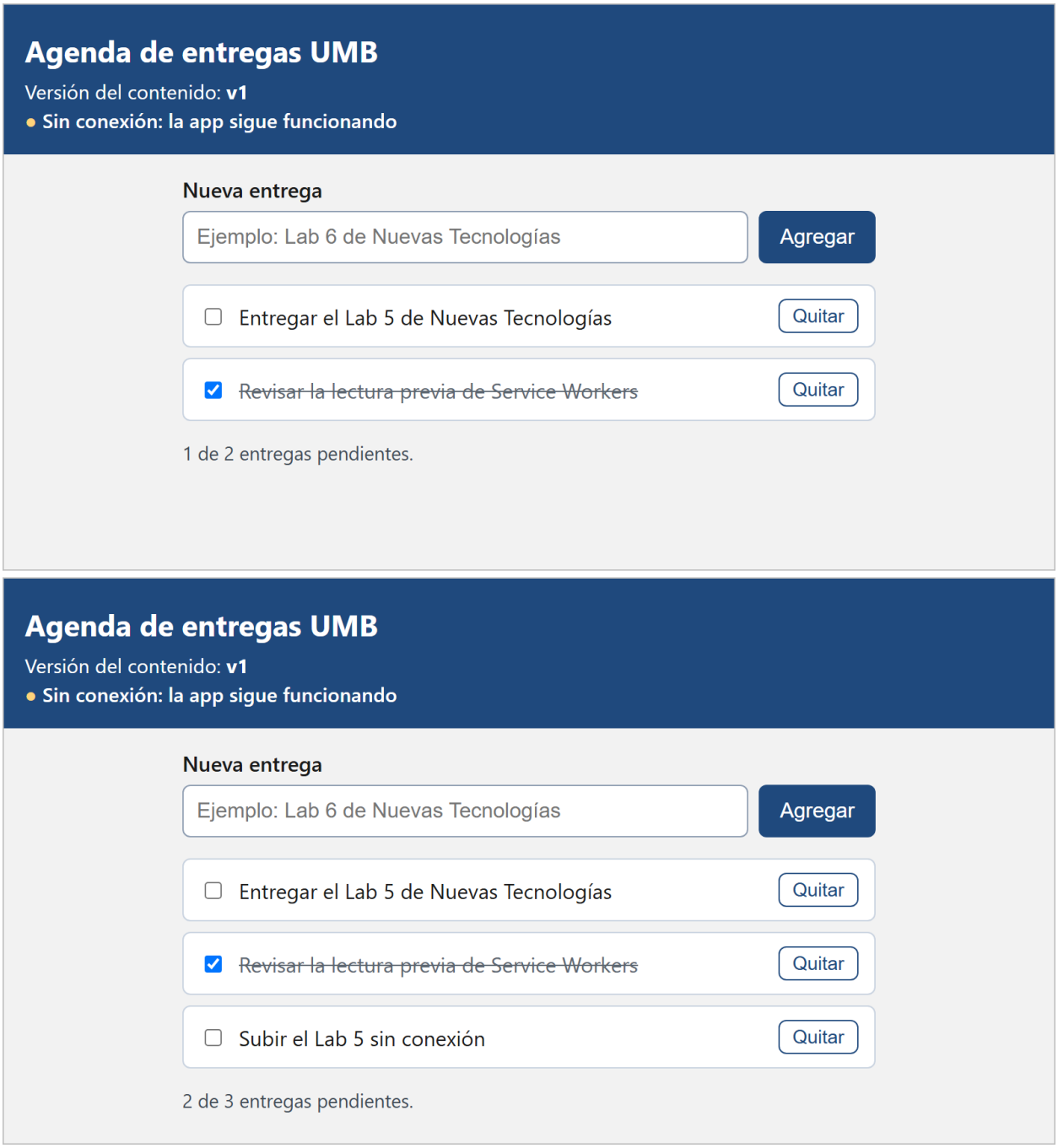
Nota. Captura real del panel Network de Chrome DevTools. Se unió la franja inferior con el resumen de solicitudes al recorte de la tabla, y se dejó entre ambas un pequeño espacio en blanco.

Además de la carga, se comprobó que la app sigue siendo útil sin conexión: con «Offline» activo se agregó la entrega «Subir el Lab 5 sin conexión», y la lista pasó de 2 a 3 entregas

(Figura 11b). Esto no depende del Service Worker, sino de que los datos viven en el navegador.

La Tabla 8 reúne las mediciones.

Figura 11
La app sin conexión: recarga y nueva entrega



Nota. Capturas reales de la pestaña de la app con «Offline» marcado: a) recarga completa, con el indicador «Sin conexión: la app sigue funcionando»; b) después de agregar una entrega. Se recortó el espacio vacío bajo cada página.

Tabla 8
Mediciones de la prueba sin conexión

Medición	Resultado
Indicador de la app con «Offline» marcado	Sin conexión: la app sigue funcionando
***navigator.onLine* con «Offline» marcado**	false
Página controlada por el Service Worker	Sí
Recursos de la página atendidos por el Service Worker	5 de 5, además del documento
Versión del contenido que se muestra	v1
Entregas antes y después de agregar una	2 y 3
Después de desmarcar «Offline» y recargar	En línea, con 3 entregas

Nota. Elaboración propia con las mediciones de evidencia/resultados.json. Los recursos atendidos se cuentan con la propiedad *workerStart* de la API de rendimiento del navegador.

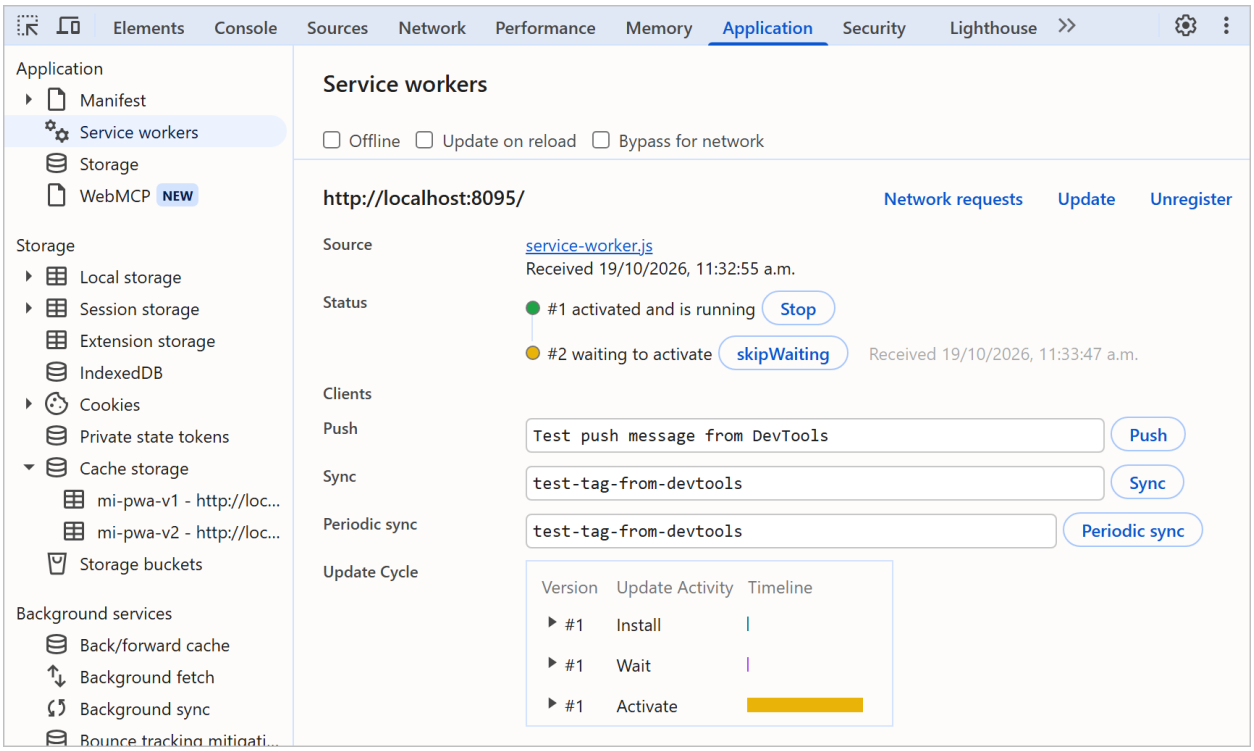
Antes de continuar se desmarcó la casilla «Offline», como pide la guía, y se verificó con el script que quedó desmarcada.

2.6 Actualizar la versión del Service Worker

La guía recuerda que el nombre del caché debe cambiar cuando se actualiza la app. Para ver qué ocurre, se cambió *CACHE_NAME* a *mi-pwa-v2* y el contenido de la página a «v2». Al pedir una actualización, el navegador instaló la versión nueva, que creó el caché *mi-pwa-v2* pero quedó «waiting to activate», porque la versión anterior seguía controlando la página: DevTools muestra las dos versiones y el botón *skipWaiting*, y en ese momento coexistían los cachés *mi-pwa-v1* y *mi-pwa-v2* (Figura 12).

Figura 12

Versión nueva del Service Worker en espera

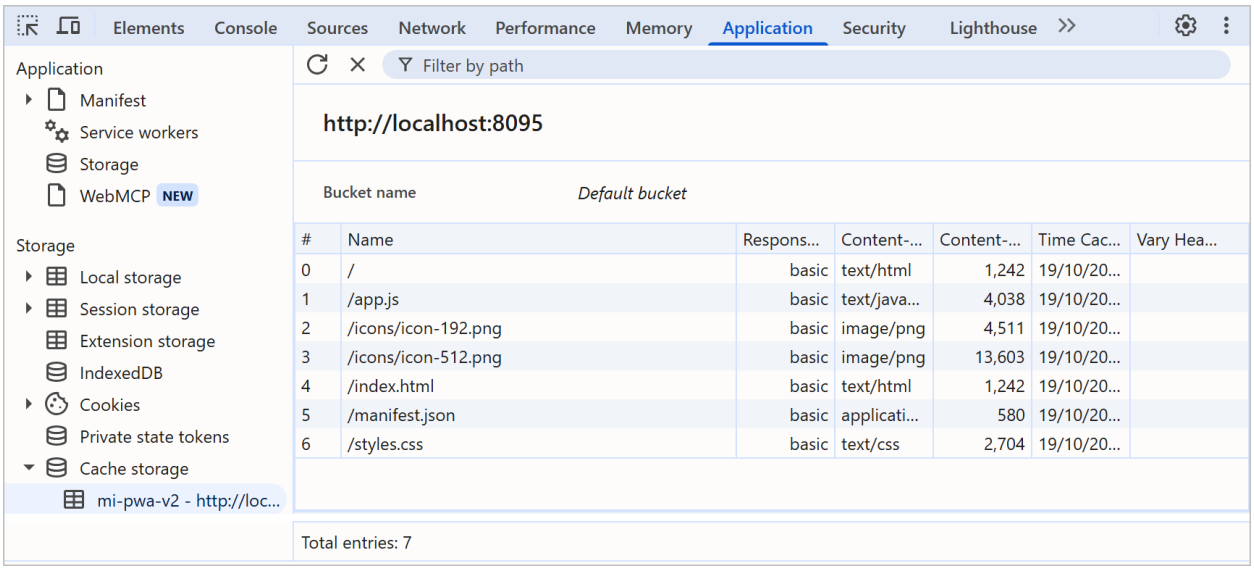


Nota. Captura real de Chrome DevTools, Application > Service workers: la versión n.º 1 activa y la n.º 2 en espera, con el botón skipWaiting.

Al saltar la espera con *skipWaiting*, la versión nueva se activó y su evento *activate* borró el caché viejo: solo quedó *mi-pwa-v2* (Figura 13). Tras recargar, la página mostró «v2». Esto demuestra para qué sirven los dos mecanismos que el código de la guía deja listos, el nombre del caché y la limpieza en *activate*, y es la forma de superar la desventaja de Cache-First que se mide en la sección 2.7.

Figura 13

Cache Storage después de actualizar: solo queda mi-pwa-v2



Nota. Captura real de Chrome DevTools, Application > Cache storage. Se unió la franja inferior con el total de entradas al recorte de la tabla.

2.7 Desafío: Network-First

El desafío pide cambiar la estrategia del evento *fetch* a Network-First: primero intenta la red y, si falla, usa el caché. Se reimplementó en *service-worker-network-first.js*, que conserva *install* y *activate* y solo cambia *fetch*. Como *app.js* registra siempre el nombre *service-worker.js* y el navegador admite un solo Service Worker por alcance, para probar esta versión se copia su contenido sobre *service-worker.js*. Figura 14 a Figura 16 muestran el archivo completo, comentado.

Figura 14

Código de service-worker-network-first.js con Network-First (parte 1 de 3)

service-worker-network-first.js

```
/*
 * SERVICE WORKER — VERSIÓN 2: NETWORK-FIRST («primero la red»)
 * ------------------------------------------------------------------------
 * Estrategia: ante cada petición, el Service Worker intenta primero la red.
 * Si la red responde, entrega esa respuesta (la más reciente) y además guarda
 * una copia en el caché. Si la red falla (sin conexión o servidor caído), usa
 * la última copia guardada en el caché.
 *
 * Ventajas:  el contenido siempre está al día cuando hay conexión, y el caché
 *            se renueva solo con cada visita.
 * Desventaja: depende de la red para responder (si la red está lenta, la
 *             página también) y no mejora el tiempo de carga.
 *
 * Esta versión solo cambia el evento FETCH respecto de service-worker.js
 * (Cache-First). INSTALL y ACTIVATE son iguales. Para probarla, su contenido se
 * copia sobre service-worker.js, porque app.js registra ese nombre y el
 * navegador solo admite un Service Worker por alcance.
 */

// Nombre del cache — cambia la version cuando actualices la app
const CACHE_NAME = 'mi-pwa-v1';

// Archivos que se guardan en cache al instalar
const ARCHIVOS_CACHE = [
  '/',
  '/index.html',
  '/styles.css',
  '/app.js',
  '/manifest.json',
  '/icons/icon-192.png',
  '/icons/icon-512.png',
];

```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker-network-first.js, líneas 1 a 33.

Figura 15

Código de service-worker-network-first.js con Network-First (parte 2 de 3)

```
34  // EVENTO INSTALL: igual que en Cache-First, guarda los archivos al instalar
35  // para que exista una copia a la cual recurrir cuando falle la red.
36  self.addEventListener('install', (event) => {
37    console.log('[SW] Instalando...');
38    event.waitUntil(
39      caches.open(CACHE_NAME).then((cache) => {
40        console.log('[SW] Guardando archivos en cache');
41        return cache.addAll(ARCHIVOS_CACHE);
42      })
43    );
44  });
45
46  // EVENTO ACTIVATE: igual que en Cache-First, limpia caches viejos.
47  self.addEventListener('activate', (event) => {
48    console.log('[SW] Activando...');
49    event.waitUntil(
50      caches.keys().then((nombres) => {
51        return Promise.all(
52          nombres.filter(n => n !== CACHE_NAME).map(n => caches.delete(n))
53        );
54      })
55    );
56  });
57
58  // EVENTO FETCH: Network-First (primero la red, si falla usa el cache).
59  self.addEventListener('fetch', (event) => {
60    // Solo se gestionan las peticiones GET: el caché no puede guardar POST, etc.
61    if (event.request.method !== 'GET') {
62      return;
63    }
64
```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker-network-first.js, líneas 34 a 64.

El evento *fetch* atiende solo las peticiones GET, porque el caché no puede guardar las demás. Pide la petición a la red y, si responde con estado 200, guarda una copia con *cache.put()* usando *clone()* y entrega la respuesta original; si la red falla, el *catch* devuelve la última copia con *caches.match()*.

Figura 16

Código de service-worker-network-first.js con Network-First (parte 3 de 3)

```
65    event.respondWith(
66      fetch(event.request)
67        .then((respuestaRed) => {
68          // La red respondió: se guarda una copia fresca en el caché.
69          // Una respuesta solo se puede leer una vez, por eso se usa clone():
70          // una copia va al caché y la original va a la página.
71          if (respuestaRed && respuestaRed.status === 200) {
72            const copia = respuestaRed.clone();
73            caches.open(CACHE_NAME).then((cache) => cache.put(event.request, copia));
74          }
75          return respuestaRed; // responde con lo más reciente
76        })
77        .catch(() => {
78          // La red falló: se usa la última copia guardada (si existe).
79          return caches.match(event.request);
80        })
81    );
82  });
83
84  /*
85   * COMPARACIÓN CON CACHE-FIRST (service-worker.js)
86   * ------------------------------------------------------------------------
87   *                        Cache-First                 Network-First (este archivo)
88   * Primer intento         caché                       red
89   * Si el primero falla    red                         caché
90   * Contenido              puede estar desactualizado  siempre el más reciente
91   * Velocidad              inmediata                   depende de la red
92   * Sin conexión           funciona                    funciona con lo ya guardado
93   * Actualiza el caché     solo al instalar            en cada respuesta de la red
94   * Conviene para          recursos que casi no cambian contenido que cambia a menudo
95   */
96
```

Nota. Captura real de Visual Studio Code 1.142.0 con el resaltado de sintaxis del editor: service-worker-network-first.js, líneas 65 a 96. Al final del archivo, un cuadro compara las dos estrategias.

La Tabla 9 resume en qué difieren las dos versiones del evento *fetch*.

Tabla 9

Diferencias entre las dos versiones del evento fetch

Aspecto	Cache-First (service-worker.js)	Network-First (service-worker-network-first.js)
Primer intento	*caches.match(event.request)*	*fetch(event.request)*
Alternativa	Si no hay copia, *fetch()* a la red.	Si la red falla, *caches.match()* con la última copia.
Escritura en el caché	Solo en *install*.	En *install* y en cada respuesta de la red con estado 200 (*cache.put*).
Uso de *clone()*	No lo necesita.	Sí: una copia va al caché y la original a la página.
Peticiones que atiende	Todas.	Solo las GET.

Nota. Elaboración propia a partir del código de las figuras de esta sección y de la sección 2.3.

Para comparar las dos estrategias con datos, el script repitió con cada una el mismo experimento: cargar la app, esperar a que el Service Worker controlara la página, cambiar la versión del contenido de «v1» a «v2» en el servidor y recargar, y después detener el servidor y recargar otra vez. La Tabla 10 reúne las mediciones; Figura 17 y Figura 18 las muestran.

Tabla 10

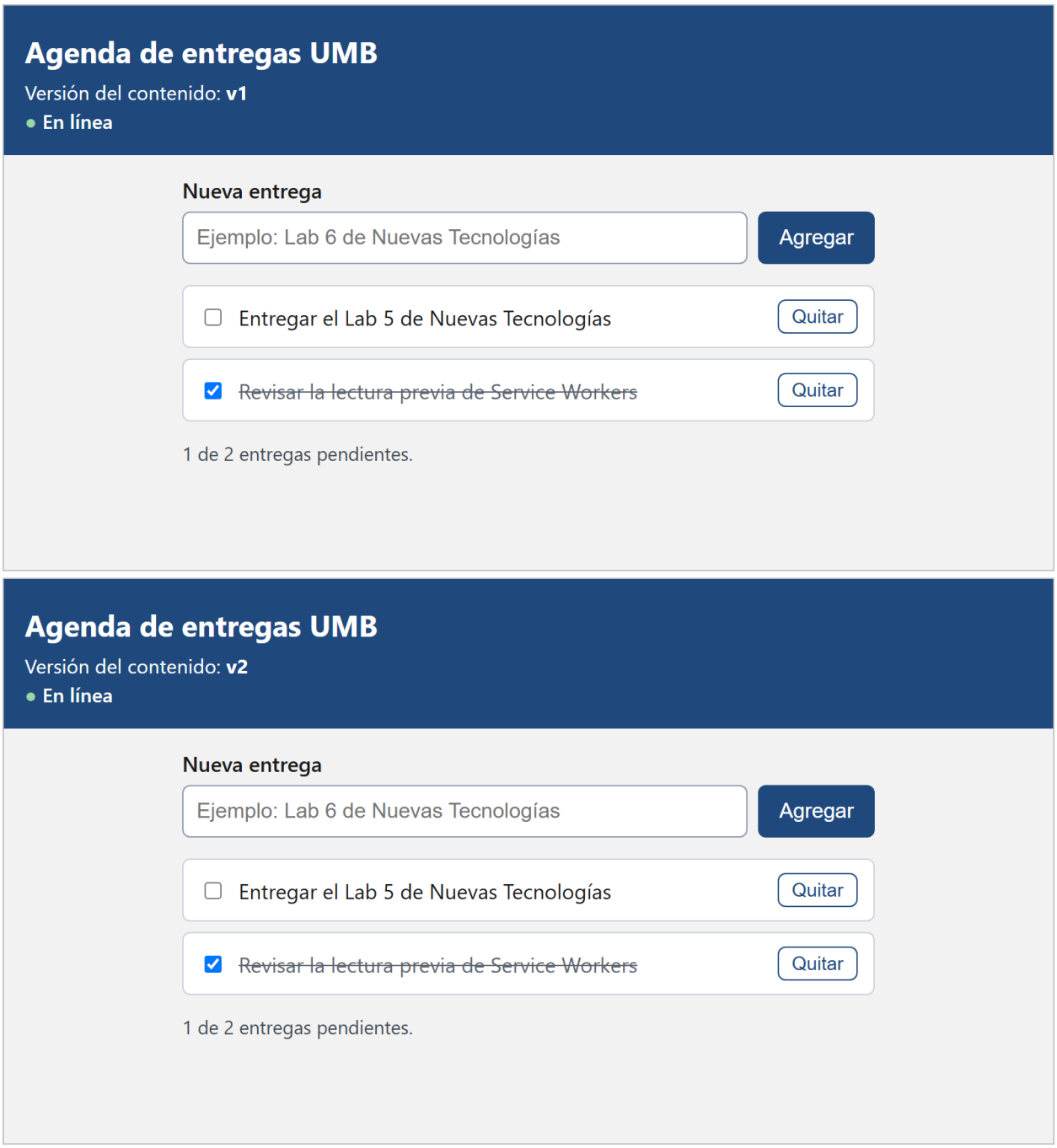
Comparación de Cache-First y Network-First con el contenido cambiado en el servidor

Medición	Cache-First	Network-First
Versión en el servidor tras el cambio	v2	v2
Versión que muestra la página al recargar	v1	v2
Versión guardada en el caché para «/» tras recargar	v1	v2
Versión que muestra en una segunda recarga	v1	v2
Con el servidor detenido: ¿carga la página?	Sí	Sí
Con el servidor detenido: versión que muestra	v1	v2

Nota. Elaboración propia con las mediciones de evidencia/resultados.json, con el contenido cambiado de v1 a v2 en el servidor.

Figura 17

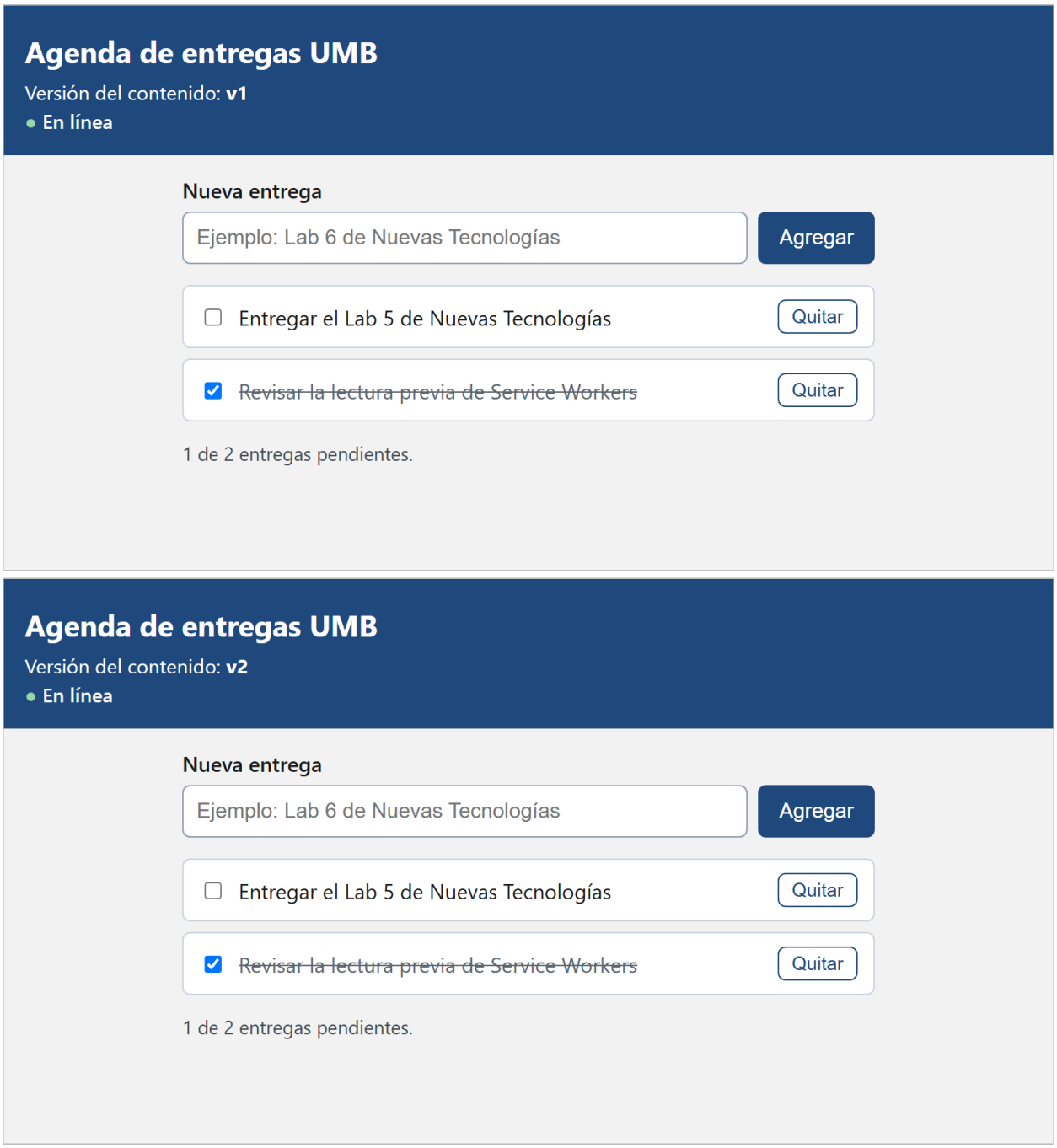
Página tras cambiar el contenido en el servidor: Cache-First y Network-First



Nota. Capturas reales de la pestaña de la app tras recargar con el servidor en v2: arriba, Cache-First (muestra v1); abajo, Network-First (muestra v2).

Figura 18

Página con el servidor detenido: Cache-First y Network-First



Nota. Capturas reales tras detener el servidor y recargar: arriba, Cache-First (v1); abajo, Network-First (v2). El indicador dice «En línea» porque el navegador tiene red; el que falló es el servidor, no la conexión.

Los resultados confirman lo que el código anticipa. Cache-First siguió mostrando «v1» aunque el servidor ya tenía «v2», y lo seguirá haciendo hasta que cambie el nombre del caché

(sección 2.6): nunca consultó la red. Network-First mostró «v2» desde la primera recarga y además actualizó la copia guardada. Con el servidor detenido, las dos estrategias cargaron la página: Cache-First con la copia que ya tenía y Network-First con la última copia que había guardado, que ya era la de «v2».

Esto matiza el enunciado de la guía, según el cual Network-First no funciona sin conexión. Funciona sin conexión con lo que ya haya guardado, porque su alternativa ante una falla de red es el caché; lo que no puede ofrecer es contenido que nunca se haya guardado. Su costo es otro: depende de la red para responder, y si esta es lenta o intermitente, el usuario espera antes de ver la copia (Archibald, 2014). Esa espera no se midió aquí, porque la red de pruebas es local.

2.8 Reflexión: qué estrategia usar y por qué

Pregunta. La estrategia Cache-First es rápida pero puede mostrar contenido desactualizado, y Network-First tiene contenido fresco pero depende de la red. ¿Cuál se usaría para (a) un blog de noticias, (b) una app de calculadora y (c) un chat en tiempo real? La Tabla 11 resume la respuesta y cada caso se justifica después.

Tabla 11
Estrategia elegida para cada caso

Caso	Estrategia	Razón principal
(a) Blog de noticias	Network-First para los artículos; Cache-First para estilos, scripts e imágenes.	El contenido cambia a menudo y debe estar al día; el aspecto casi no cambia.
(b) App de calculadora	Cache-First.	Todo el código viaja con la app y nada cambia entre usos; responde de inmediato y sin red.
(c) Chat en tiempo real	Solo red para los mensajes; Cache-First para la interfaz.	Un mensaje en caché es un mensaje viejo; lo que sí puede guardarse es la pantalla.

Nota. Elaboración propia con base en las mediciones de las secciones 2.6 y 2.7, Archibald (2014) y Chrome for Developers (2021).

(a) Blog de noticias. Se usaría Network-First para el contenido de los artículos y la portada. Quien abre un blog de noticias quiere lo último, y la prueba de la sección 2.7 muestra el problema de la alternativa: con Cache-First, el lector habría seguido viendo «v1» aunque el

servidor ya publicara «v2». Con Network-First ve lo más reciente cuando hay conexión y, sin ella, puede releer los artículos que ya había abierto. Es la recomendación para páginas HTML y respuestas de API (Chrome for Developers, 2021) y para contenido como los artículos (Archibald, 2014). Su riesgo es que una red lenta retrasa la página; una opción es stale-while-revalidate, que muestra la copia de inmediato y la renueva para la próxima visita, aceptable si la última noticia no es crítica. Los estilos, scripts e imágenes versionados sí van con Cache-First, porque no cambian.

(b) App de calculadora. Cache-First. Una calculadora no depende de datos del servidor: su lógica y su interfaz son archivos estáticos que se guardan al instalar. Con Cache-First abre al instante, sin red, y el riesgo de contenido desactualizado casi no existe, porque lo único que cambia son las versiones de la propia app; para esas basta cambiar el nombre del caché, como se hizo en la sección 2.6. Es el caso típico de las aplicaciones que funcionan primero sin conexión y de los recursos estáticos (Archibald, 2014; Chrome for Developers, 2021).

(c) Chat en tiempo real. Ninguna de las dos estrategias sirve para los mensajes en sí. Un chat vale por lo que acaba de pasar, y una respuesta guardada en el caché es, por definición, una foto vieja: mostrarla como si fuera la conversación actual es peor que no mostrarla. Los mensajes deberían pedirse siempre a la red, sin caché (estrategia «solo red»), y la interfaz, que sí es estática, se serviría con Cache-First para que la pantalla abra aunque no haya conexión, con un aviso de que los mensajes no se pueden actualizar. Si hubiera que elegir solo entre las dos de la guía, sería Network-First, con el caché como último recurso para mostrar el historial ya recibido. Para avisar de mensajes nuevos con la app cerrada, la propia documentación propone los mensajes push (Archibald, 2014), que esta guía menciona pero no pide.

Idea común. La elección no se hace por aplicación, sino por tipo de recurso: lo que no cambia se sirve del caché; lo que cambia, de la red. Un mismo Service Worker puede aplicar estrategias distintas según la ruta de cada petición, como en el blog, donde los artículos y los estilos siguen reglas diferentes.

Conclusiones

Un Service Worker es un intermediario entre la app y la red que se instala una vez, se activa cuando la versión anterior deja de usarse y desde entonces decide cómo responder cada petición. El de la guía, con Cache-First, quedó activo, sin errores en la consola, y con un caché de siete entradas que coincide con la lista de archivos del código; la primera carga no quedó controlada y las siguientes sí.

Con la casilla «Offline» marcada, la página recargada se sirvió completa desde el Service Worker: 5 recursos más el documento, con 0 B transferidos. Al actualizar la versión se observó la fase de espera y cómo *activate* deja solo el caché nuevo.

El desafío mostró la diferencia entre las dos estrategias con datos: ante un cambio en el servidor, Cache-First siguió mostrando la versión guardada y Network-First mostró la nueva y actualizó su copia; con el servidor detenido, las dos cargaron la página. Por eso la elección depende del tipo de contenido: caché para lo que no cambia, red para lo que debe estar al día.

Las limitaciones del trabajo son tres. Las notificaciones push no se implementaron porque la guía no las pide. Las pruebas corrieron en un solo navegador, Chrome 155 sin ventana, con un servidor local; no se midió la latencia con redes lentas ni se probó en otros navegadores. Y la actualización de versión se activó con el comando *skipWaiting* del protocolo de DevTools y no con un clic en el botón.

Referencias

Archibald, J. (2014, 9 de diciembre). *Common techniques to build offline applications*. web.dev. https://web.dev/articles/offline-cookbook

Archibald, J. (2016, 29 de septiembre). *The service worker lifecycle*. web.dev. https://web.dev/articles/service-worker-lifecycle

Basques, K., y Emelianova, S. (2016, 25 de julio). *Debug Progressive Web Apps*. Chrome for Developers. https://developer.chrome.com/docs/devtools/progressive-web-apps

Chrome for Developers. (2021, 24 de septiembre). *Strategies for service worker caching*. Workbox. https://developer.chrome.com/docs/workbox/caching-strategies-overview

Google. (s. f.). *Google Chrome* (Versión 155) [Software]. https://www.google.com/chrome/

MDN Web Docs. (s. f.-a). *Service Worker API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

MDN Web Docs. (s. f.-b). *Using Service Workers*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers

Microsoft. (s. f.). *Visual Studio Code* (Versión 1.142.0) [Software]. https://code.visualstudio.com

Universidad Manuela Beltrán. (2026). *Guía de laboratorio n.º 5: Service Worker* [Guía de laboratorio, módulo 2: PWA, Nuevas Tecnologías del Desarrollo]. UMB.